improvement at -2, and 0.0036 at 3. Looking at Figure 4.10, we see that these numbers make sense. Not only at these three points (0, -2, and 3), but we can also compute how likely a given point is able to improve, that is, the *probability of improvement* of a given point, across our search space, using the same formula.

IMPORTANT **The probability of improvement score**

The procedure in Figure 4.11 gives us the scoring rule for the Probability of Improvement (PoI) policy, where the score of each point in the search space is equal to how likely the point can improve from the incumbent. Again, this score is also called the acquisition score, as we are using it as a method of data acquisition.

We can realize this PoI policy using the `cdf()` method, but it's cleaner to leverage BoTorch, the Python library that implements BayesOpt policies, for this task. BoTorch is built on top of and works seamlessly with PyTorch and GPyTorch. As we saw earlier, we only needed to change two lines of code in our GP class to make the model compatible with BoTorch. Further, BoTorch implements its policies as *modules*, allowing us to swap different policies in and out of a BayesOpt loop in a modular manner.

SIDEBAR **The modularity of BoTorch's policies**

By "modular", we mean that we can replace the policy currently in use with another policy in a BayesOpt loop by only changing the initialization of the policy. The rest of the BayesOpt loop (training the Gaussian process model, visualizing optimization progress, updating the training data) doesn't have to change. We have seen a similar modularity with GPyTorch's mean functions and kernels in Sections 3.3 and 3.4.2.

To implement the PoI policy with BoTorch, we do the following:

```
policy = botorch.acquisition.analytic.ProbabilityOfImprovement(   ❶
    model, best_f=train_y.max()   ❶
)   ❶

with torch.no_grad():   ❷
    scores = policy(xs.unsqueeze(1))   ❷
```

❶ Declare the Probability of Improvement policy.

❷ Compute the score.

The BoTorch class `ProbabilityOfImprovement` implements PoI as a PyTorch module, taking in a GP as the first argument and the incumbent value as the second. The variable `scores` now stores the PoI scores of the points in `xs`, which is a dense grid between -5 and 5.

Again, the acquisition score of each point equals the probability that the point will improve from

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>